Respostas

Pergunta 1

a) O telnet é um utilitário baseado num protocolo que permite a comunicação de texto bidireccional e interactiva através de uma consola virtual, concebido para acesso remoto a uma shell. Funciona por cima do TCP. Por razões de segurança, foi substituído pelo ssh.

b) Ver, por exemplo, mars.netanya.ac.il/~unesco/cdrom/booklet/HTML/NETWORKING/node300.html

O telnet estabelece uma conexão TCP para uma porta do servidor remoto (porta 23, a menos que outra seja explicitamente indicada). Nessa conexão, circulam intercalados texto e informação de controlo (sequências de caracteres).

Utilização:

```
telnet [options] host [port]
```

c) Ligar ao servidor:

```
telnet neverssl.com 80
```

Fazer copy/paste dos pedidos HTTP e observar o que acontece.

d) No primeiro caso, o servidor fecha a conexão após devolver a resposta.

No segundo caso, a conexão é mantida aberta por um certo tempo, sendo possível enviar novos pedidos (conexão persistente).

No terceiro caso são enviados dois pedidos em rajada (conexão persistente com pipelining).

No último caso é solicitado ao servidor o fecho da conexão após devolver a resposta (conexão não persistente, mesmo sendo HTTP 1.1).

Pergunta 2

a) Atraso total sem proxy: $\mathbf{n \cdot (2RTT + d_{trans})}$

Atraso total com proxy: $\mathbf{RTT + n \cdot (RTT + d_{trans})}$

Poupança: $\mathbf{(n-1) \cdot RTT}$

Nota: $RTT = 2 \cdot d_{prop}$

b) RTT_{BP} entre o browser e o proxy, RTT_{PS} entre o proxy e o servidor. RTT_{PS} será semelhante ao RTT da alínea anterior.

Assumindo que a capacidade entre o browser e o proxy não é inferior à capacidade entre o proxy e o servidor:

Atraso total: $\mathbf{2n \cdot RTT_{BP} + (n+1) \cdot RTT_{PS} + n \cdot d_{trans}}$

Se $RTT_{BP} \ll RTT_{PS}$ (e.g., está na rede local), há uma poupança ligeiramente menor do que no caso anterior.

Se $RTT \approx RTT_{PS}$, demora ainda mais do que sem proxy.

c) Neste caso poderia tirar-se partido do pipelining.

Atraso total: **$2 \cdot RTT + n \cdot d_{trans}$**

Poupança: **$(2n-2) \cdot RTT$**

(Com o browser a falar HTTP 1.1, com ou sem proxy na máquina local é o mesmo.)

Pergunta 3

Ligar ao servidor SMTP:

```
gnutls-cli smtp.fc.up.pt -p 25 --starttls-proto=smtp --crlf
```

E depois enviar:

```
HELO warnerbros.com
MAIL FROM: <bugsbunny@warnerbros.com>
RCPT TO: <up123456789@edu.fc.up.pt>
DATA
From: Bugs Bunny <bugsbunny@warnerbros.com>
To: up123456789@edu.fc.up.pt
Subject: Eh... What's up, doc?

Did you know email is insecure?
.
QUIT
```

Pergunta 4

a)

```
host www.dcc.fc.up.pt
www.dcc.fc.up.pt has address 193.136.39.12
```

b)

```
dig www.google.com
```

4 secções (question, answer, authority, additional); ver respectivo conteúdo.

c)

```
dig +norecurse www.dcc.fc.up.pt @198.41.0.4
dig +norecurse www.dcc.fc.up.pt @204.61.216.105
dig +norecurse www.dcc.fc.up.pt @193.137.55.33
```

d)

```
dig +trace www.dcc.fc.up.pt
```

e)

```
dig www.jn.pt
```

f) 4 endereços; o primeiro da lista vai rodando. Pelo facto de haver mais do que um, obtemos tolerância a falhas. Como as aplicações normalmente começam por usar o primeiro da lista, obtemos também distribuição de carga.

g)

```
dig fc.up.pt MX
```

h) O campo "id" é igual no pedido e na respectiva resposta, mas é diferente em pedidos diferentes. Permite associar uma resposta a um pedido mesmo não havendo conexões (o DNS funciona normalmente sobre UDP).

Pergunta 5

a) O TTL vai diminuindo; quando chega a zero é reposto no valor inicial e volta a diminuir.

b) Neste caso o TTL tem sempre o mesmo valor. O TTL define o tempo que resta em cache. Quando o pedido é feito ao servidor com autoridade, tem sempre o valor máximo. Quando é feito a uma cache (servidor local) vai diminuindo; quando chega a zero o servidor local consulta novamente o servidor com autoridade e coloca em cache a resposta "fresca".

Pergunta 6

a) Pedir os registos NS para a raiz (.)

b)

```
dig . NS
```

Pergunta 7

Experimentar o que é pedido e observar.